

Problem 1: Circle Class

Objectives:

- To create a Circle class with radius and area
- To initialize multiple circles and calculate their areas and total area

Theory:

A circle is defined by its radius and its area can be calculated using the formula $\text{Area} = \pi * r * r$. OOP allows the radius and area calculation method to be combined into a single class. This makes it easier to manage multiple circles and perform operations on each object separately

Code:

```
Lab 1 > C++ 1.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class Circle
5  {
6  private:
7      int radius;
8
9  public:
10     Circle(int r) : radius(r) {} // Constructor to initializer list
11
12     float getArea() const
13     {
14         return 3.14159 * radius * radius; // To calculate area
15     }
16 };
17
18 int main()
19 {
20     Circle c1(2), c2(3), c3(5);
21
22     float total = c1.getArea() + c2.getArea() + c3.getArea();
23
24     cout << "Area of circle 1: " << c1.getArea() << endl;
25     cout << "Area of circle 2: " << c2.getArea() << endl;
26     cout << "Area of circle 3: " << c3.getArea() << endl;
27     cout << "Total area of three circles: " << total << endl;
28
29     return 0;
30 }
```

Procedure:

1. The Circle class was created with radius as a private member
2. A method to calculate area was written
3. Three Circle objects were initialized in main
4. Individual areas and total area were calculated and displayed

Output:

```
Area of circle 1: 12.5664
Area of circle 2: 28.2743
Area of circle 3: 78.5397
Total area of three circles: 119.38
```

Precautions:

- Radius should be positive
- Formula should be correctly implemented

Conclusion:

The area of each circle and the total area were successfully calculated using the Circle class. Object-oriented programming was demonstrated effectively by handling multiple circle objects with their own data and methods

Problem 2: Account Class

Objectives:

- To create an Account class to simulate banking operations
- To perform deposit, withdrawal, transfer and display balances

Theory:

A bank account has an account number and a balance. Deposit increases the balance, withdrawal decreases it and transfer moves money from one account to another

Code:

```
Lab 1 > C++ 2.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class Account
5  {
6  private:
7      int number;
8      int amount;
9
10 public:
11     // Constructor to initialize list
12     Account(int num, int amt) : number(num), amount(amt) {}
13
14     // To deposit money
15     void deposit(int x)
16     {
17         amount += x;
18         cout << "Deposited " << x << " to Account " << number << endl;
19     }
20
21     // To withdraw money with balance check
22     void withdraw(int x)
23     {
24         if (x <= amount)
25         {
26             amount -= x;
27             cout << "Withdrawn " << x << " from Account " << number << endl;
28         }
29         else
30         {
31             cout << "Insufficient balance in Account " << number << endl;
32         }
33     }
34
35     // To transfer money to another account
36     void transfer(Account &to, int x)
37     {
38         if (x <= amount)
39         {
40             amount -= x;
41             to.amount += x;
42             cout << "Transferred " << x << " from Account " << number
43                 << " to Account " << to.number << endl;
44         }
45         else
46         {
47             cout << "Insufficient balance in Account " << number
48                 << " for transfer" << endl;
49         }
50     }
51
52     // To display account details
53     void show() const
54     {
55         cout << "Account No: " << number << " | Balance: " << amount << endl;
56     }
57 };
58
59 int main()
60 {
61     // To initialize 5 accounts
62     Account a1(2403176, 20);
63     Account a2(2403177, 3000);
64     Account a3(2403178, 7000);
65     Account a4(2403179, 2000);
66     Account a5(2403180, 10000);
67
68     cout << "Initial Account Balances:\n";
69     a1.show();
70     a2.show();
71     a3.show();
72     a4.show();
73     a5.show();
74
75     // To perform some transactions
76     a1.deposit(1000);
77     a2.withdraw(200);
78     a3.transfer(a4, 500);
79
80     cout << "Final Account Balances:\n";
81     a1.show();
82     a2.show();
83     a3.show();
84     a4.show();
85     a5.show();
86
87     return 0;
88 }
```

Procedure:

1. The Account class was created with account number and balance
2. Deposit, withdrawal, transfer and display methods were implemented
3. Accounts were initialized in main
4. Transactions were performed and final balances were displayed

Output:

```
Initial Account Balances:
Account No: 2403176 | Balance: 20
Account No: 2403177 | Balance: 3000
Account No: 2403178 | Balance: 7000
Account No: 2403179 | Balance: 2000
Account No: 2403180 | Balance: 10000
Deposited 1000 to Account 2403176
Withdrew 200 from Account 2403177
Transferred 500 from Account 2403178 to Account 2403179
Final Account Balances:
Account No: 2403176 | Balance: 1020
Account No: 2403177 | Balance: 2800
Account No: 2403178 | Balance: 6500
Account No: 2403179 | Balance: 2500
Account No: 2403180 | Balance: 10000
```

Precautions:

- Ensure withdrawal or transfer amount $\leq$ balance
- IDs should be unique

Conclusion:

Deposits, withdrawals and transfers were performed successfully. The Account class allowed secure and organized management of balances, demonstrating object-oriented principles in a simple banking system

Problem 3: Triangle Class

Objectives:

- To create a Triangle class
- To initialize edges, check validity and calculate the area

Theory:

A triangle is defined by its three edges. A triangle is valid if the sum of any two edges is greater than the third edge. The area can be calculated using the formula: $s = (a+b+c)/2$ and $\text{Area} = \sqrt{s(s-a)(s-b)(s-c)}$. Using a class allows both the edges and the methods for validation and area calculation to be combined in one structure

Code:

```

Lab 1 > C++ 3.cpp > [main]
1 #include <iostream>
2 #include <cmath> // for sqrt
3 using namespace std;
4
5 class Triangle
6 {
7 private:
8     int edge1, edge2, edge3;
9     float area;
10
11 public:
12     // Constructor to initialize edges
13     Triangle(int e1, int e2, int e3) : edge1(e1), edge2(e2), edge3(e3), area(0) {}
14
15     // To check if the edges can form a valid triangle
16     bool isValid() const
17     {
18         return (edge1 + edge2 > edge3) &&
19             (edge1 + edge3 > edge2) &&
20             (edge2 + edge3 > edge1);
21     }
22
23     // To calculate area
24     void calculateArea()
25     {
26         if (isValid())
27         {
28             float s = (edge1 + edge2 + edge3) / 2.0;
29             area = sqrt(s * (s - edge1) * (s - edge2) * (s - edge3));
30         }
31         else
32         {
33             area = 0;
34         }
35     }
36
37     // To display the area
38     void showArea() const
39     {
40         if (area > 0)
41             cout << "Area of the triangle: " << area << endl;
42         else
43             cout << "Cannot calculate area. Invalid triangle edges." << endl;
44     }
45
46     // To display the edges
47     void showEdges() const
48     {
49         cout << "Edges: " << edge1 << ", " << edge2 << ", " << edge3 << endl;
50     }
51 };
52
53 int main()
54 {
55     // To initialize a triangle
56     Triangle t(3, 4, 5);
57
58     cout << "Triangle edges:\n";
59     t.showEdges();
60
61     // To check validity
62     if (t.isValid())
63     {
64         cout << "The edges form a valid triangle.\n";
65         t.calculateArea();
66         t.showArea();
67     }
68     else
69     {
70         cout << "The edges do NOT form a valid triangle.\n";
71     }
72
73     return 0;
74 }

```

Procedure:

1. The Triangle class was created with three edges as private members
2. Validity was checked using the triangle inequality rule
3. Area was calculated and displayed for valid triangles

Output:

```

Triangle edges:
Edges: 3, 4, 5
The edges form a valid triangle.
Area of the triangle: 6

```

Precautions:

- Edges must be positive
- Check validity before calculating area

Conclusion:

The Triangle class successfully verified triangle validity and calculated its area. Object-oriented principles allowed encapsulation of edges and operations efficiently

Problem 4: Gym Member

Objectives:

- To create a Member class for gym members
- To calculate BMI and classify members' health status
- To find min, max and average height & weight

Theory:

Each member has an ID, height and weight. BMI is calculated using the formula $BMI = \text{Weight} / \text{Height}^2$ and the member is classified as underweight, normal, overweight, or obese according to standard BMI ranges. Classes make it easy to handle multiple members and perform operations like min, max, average and BMI classification

Code:

```
Lab 1 > C++ 4.cpp > main()
1  #include <iostream>
2  #include <vector>
3  #include <limits>
4  using namespace std;
5
6  // Structure to store Member data
7  struct Member {
8      int id;
9      float height; // meters
10     float weight; // kg
11
12     // To calculate BMI
13     float calculateBMI() const {
14         return weight / (height * height);
15     }
16
17     // To display BMI
18     void displayBMI() const {
19         float bmi = calculateBMI();
20         cout << "Member ID: " << id << " | BMI: " << bmi << " | Classification: ";
21         if (bmi < 16) cout << "Severe Thinness";
22         else if (bmi < 17) cout << "Moderate Thinness";
23         else if (bmi < 18.5) cout << "Mild Thinness";
24         else if (bmi < 25) cout << "Normal";
25         else if (bmi < 30) cout << "Overweight";
26         else if (bmi < 35) cout << "Obese Class I";
27         else if (bmi < 40) cout << "Obese Class II";
28         else cout << "Obese Class III";
29         cout << endl;
30     }
31 };
32
33 // Global vector to store members
34 vector<Member> members;
35
36 // Function to add a member
37 void addMember() {
38     Member m;
39     cout << "Enter Member ID: ";
40     cin >> m.id;
41     cout << "Enter Height (m): ";
42     cin >> m.height;
43     cout << "Enter Weight (kg): ";
44     cin >> m.weight;
45     members.push_back(m);
46     cout << "Member added successfully.\n";
47 }
48
49 // Function to update a member
50 void updateMember() {
51     int id;
52     cout << "Enter Member ID to update: ";
53     cin >> id;
54     for (auto &m : members) {
55         if (m.id == id) {
56             cout << "Enter new Height (m): ";
57             cin >> m.height;
58             cout << "Enter new Weight (kg): ";
59             cin >> m.weight;
60             cout << "Member updated successfully.\n";
61             return;
62         }
63     }
64     cout << "Member not found.\n";
65 }
66
67 // Function to remove a member
68 void removeMember() {
69     int id;
70     cout << "Enter Member ID to remove: ";
71     cin >> id;
72     for (auto it = members.begin(); it != members.end(); ++it) {
73         if (*it.id == id) {
74             members.erase(it);
75             cout << "Member removed successfully.\n";
76             return;
77         }
78     }
79     cout << "Member not found.\n";
80 }
81
82 // Function to calculate max height and weight
83 void maxHeightWeight() {
84     if (members.empty()) {
85         cout << "No members available.\n";
86         return;
87     }
88     float maxHeight = members[0].height;
89     float maxWeight = members[0].weight;
90     for (const auto &m : members) {
91         if (m.height > maxHeight) maxHeight = m.height;
92         if (m.weight > maxWeight) maxWeight = m.weight;
93     }
94     cout << "Max Height: " << maxHeight << " m | Max Weight: " << maxWeight << " kg\n";
95 }
96
97 // Function to calculate min height and weight
98 void minHeightWeight() {
99     if (members.empty()) {
100         cout << "No members available.\n";
101         return;
102     }
103     float minHeight = members[0].height;
104     float minWeight = members[0].weight;
105     for (const auto &m : members) {
106         if (m.height < minHeight) minHeight = m.height;
107         if (m.weight < minWeight) minWeight = m.weight;
108     }
109     cout << "Min Height: " << minHeight << " m | Min Weight: " << minWeight << " kg\n";
110 }
111
112 // Function to calculate average height and weight
113 void averageHeightWeight() {
114     if (members.empty()) {
115         cout << "No members available.\n";
116         return;
117     }
118     float sumH = 0, sumW = 0;
119     for (const auto &m : members) {
120         sumH += m.height;
121         sumW += m.weight;
122     }
123     cout << "Average Height: " << sumH / members.size() << " m | Average Weight: " << sumW / members.size() << " kg\n";
124 }
125
126 // Function to display BMI of all members
127 void displayBMI() {
128     if (members.empty()) {
129         cout << "No members available.\n";
130         return;
131     }
132     for (const auto &m : members) {
133         m.displayBMI();
134     }
135 }
136
137 int main() {
138     int choice;
139     do {
140         cout << "\n***** Main Menu *****\n";
141         cout << "1. Add Member\n";
142         cout << "2. Update Member\n";
143         cout << "3. Remove Member\n";
144         cout << "4. Max Height and Weight\n";
145         cout << "5. Min Height and Weight\n";
146         cout << "6. Average Height and Weight\n";
147         cout << "7. BMI Classification\n";
148         cout << "8. Exit\n";
149         cout << "Enter your option (1-8): ";
150         cin >> choice;
151
152         switch (choice) {
153             case 1: addMember(); break;
154             case 2: updateMember(); break;
155             case 3: removeMember(); break;
156             case 4: maxHeightWeight(); break;
157             case 5: minHeightWeight(); break;
158             case 6: averageHeightWeight(); break;
159             case 7: displayBMI(); break;
160             case 8: cout << "Exiting program.\n"; break;
161             default: cout << "Invalid option. Try again.\n";
162         }
163     } while (choice != 8);
164
165     return 0;
166 }
```

Procedure:

1. The Member class was created with ID, height and weight
2. Members were added, updated and removed from the system
3. BMI was calculated and classification was displayed
4. Min, max and average height & weight were calculated and displayed

Output:

```
***** Main Menu *****
1. Add Member
2. Update Member
3. Remove Member
4. Max Height and Weight
5. Min Height and Weight
6. Average Height and Weight
7. BMI Classification
8. Exit
Enter your option (1-8): █
```

Precautions:

- Enter positive height and weight
- IDs must be unique

Conclusion:

BMI calculations and member management were successfully implemented. Object-oriented programming was used to organize multiple members efficiently and statistical operations were applied accurately

Problem 1: Test Class

Objectives:

- To create a Test class with multiple constructors
- To track the total number of objects created
- To perform operations using object data members

Theory:

In object-oriented programming, constructors are used to initialize objects in different ways such as default, parameterized and copy constructors. A static data member is shared among all objects of a class and is useful for counting how many objects are created. Encapsulation ensures that data members remain protected while methods control access to them

Code:

```
Lab 2 > C++ P1.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class Test {
5  private:
6      int x, y;
7      static int z;  // To count total objects
8
9  public:
10     // Empty constructor
11     Test() {
12         x = 0;
13         y = 0;
14         z++;
15     }
16
17     // Parameterized constructor
18     Test(int a, int b) {
19         x = a;
20         y = b;
21         z++;
22     }
23
24     // Copy constructor
25     Test(const Test &t) {
26         x = t.x;
27         y = t.y;
28         z++;
29     }
30
31     // Method to initialize x and y
32     void setData(int a, int b) {
33         x = a;
34         y = b;
35     }
36
37     // To display only z
38     static void showZ() {
39         cout << "Total objects: " << z << endl;
40     }
41
42     // To display x, y, z
43     void showData() const {
44         cout << "x = " << x << ", y = " << y << ", z = " << z << endl;
45     }
46
47     int getX() const { return x; }
48     int getY() const { return y; }
49
50     // Destructor
51     ~Test() {}
52 };
53
54 int Test::z = 0;
55
56 int main() {
57     Test t1, t2(3,4), t3(t2), t4(5,6), t5(1,9);
58
59     Test::showZ();
60
61     int sumX = t1.getX() + t2.getX() + t3.getX() + t4.getX() + t5.getX();
62     cout << "Sum of x: " << sumX << endl;
63
64     Test arr[5] = {t1, t2, t3, t4, t5};
65     int maxY = arr[0].getY(), index = 0;
66
67     for(int i=1; i<5; i++) {
68         if(arr[i].getY() > maxY) {
69             maxY = arr[i].getY();
70             index = i;
71         }
72     }
73
74     cout << "Object number with max y: " << index+1 << endl;
75     return 0;
76 }
```

Procedure:

1. The Test class was created with private data members x, y and static z
2. Constructors were implemented to initialize data in different ways
3. Five objects were created using different constructors
4. The sum of x values and maximum y value were calculated
5. Destructor was defined for object cleanup

Output:

```
Total objects: 5
Sum of x: 12
Object number with max y: 5
```

Precautions:

- Static variable must be defined outside the class
- Copy constructor should receive object by reference
- Constant methods should not modify data members

Conclusion:

The Test class successfully demonstrated the use of constructors, static data members and object operations. Object counting and data analysis were performed correctly, showing proper use of object-oriented programming concepts

Problem 2: Copy Constructor

Objectives:

- To create a class with two data members
- To use setter method and copy constructor

Theory:

A copy constructor is used to create a new object by copying data from an existing object. Setter methods help initialize private data members safely. This approach ensures controlled access and proper duplication of object data

Code:

```
Lab 2 > C++ P2.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  private:
6      int a, b;
7
8  public:
9      void setData(int x, int y) {
10         a = x;
11         b = y;
12     }
13
14     // Copy constructor
15     A(const A &obj) {
16         a = obj.a;
17         b = obj.b;
18     }
19
20     A() {}
21
22     void show() const {
23         cout << "a = " << a << ", b = " << b << endl;
24     }
25 };
26
27 int main() {
28     A obj1;
29     obj1.setData(10, 20);
30
31     A obj2(obj1); // Copy constructor call
32
33     obj1.show();
34     obj2.show();
35
36     return 0;
37 }
```

Procedure:

1. The class A was created with two private data members
2. Setter method was used to assign values to obj1
3. Copy constructor was used to copy data from obj1 to obj2
4. Values were displayed for verification

Output:

```
a = 10, b = 20  
a = 10, b = 20
```

Precautions:

- Copy constructor must accept reference parameter
- Setter method should be used before copying data
- Private members should not be accessed directly

Conclusion:

The program successfully demonstrated the use of setter methods and copy constructors. Object data was copied correctly, ensuring safe and structured duplication of objects

Problem 1: Father → Son → GrandSon Inheritance

Objectives:

- To create a multi-level inheritance hierarchy: Father → Son → GrandSon
- To explore access to private, protected and public members in derived classes
- To calculate the sum of accessible members in GrandSon class

Theory:

Inheritance allows a class to acquire members and methods from another class. Access specifiers (private, protected, public) control which members are accessible in derived classes. Multi-level inheritance demonstrates how access modifiers affect visibility across different levels of hierarchy. By testing various inheritance types, one can see which members are accessible in Son and GrandSon classes and how sums of accessible members can be calculated

Code:

```
Lab 3 > C++ 1.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  // Father class
5  class Father {
6  private:
7      int money;
8  protected:
9      int gold;
10 public:
11     int land;
12
13 public:
14     Father() { money = 100; gold = 50; land = 200; } // To initialize values
15 };
16 // Son class inheriting from Father
17 class Son : public Father { // To change public into protected/private
18 public:
19     void showSon() {
20         // money; // Not accessible
21         cout << "gold in Son: " << gold << endl; // Accessible
22         cout << "land in Son: " << land << endl; // Accessible
23     }
24 };
25 // GrandSon class inheriting from Son
26 class GrandSon : public Son { // To change public into protected/private
27 public:
28     void showGrandSon() {
29         // money; // Not accessible
30         cout << "gold in GrandSon: " << gold << endl;
31         cout << "land in GrandSon: " << land << endl;
32         int sum = gold + land; // money not accessible
33         cout << "Sum of accessible members: " << sum << endl;
34     }
35 };
36 int main() {
37     Son s;
38     GrandSon gs;
39     s.showSon();
40     gs.showGrandSon();
41     return 0;
42 }
```

Procedure:

1. The Father class was created with private, protected and public members
2. The Son class was derived from Father with public inheritance
3. The GrandSon class was derived from Son with public inheritance
4. Accessible members in Son and GrandSon classes were tested and displayed
5. Sum of accessible members in GrandSon class was calculated
6. Different inheritance types were tested by modifying access specifiers

Output:

```
gold in Son: 50
land in Son: 200
gold in GrandSon: 50
land in GrandSon: 200
Sum of accessible members: 250
```

- **Son class:** gold and land accessible; money not accessible
- **GrandSon class:** gold and land accessible; money not accessible
- Sum of accessible members in GrandSon class displayed correctly

Class		In Son Class			In GrandSon Class		
Son	GrandSon	money	gold	land	money	gold	land
public	public	×	✓	✓	×	✓	✓
protected	public	×	✓	×	×	✓	×
private	public	×	×	×	×	×	×
public	protected	×	✓	✓	×	✓	×
protected	protected	×	✓	×	×	✓	×
private	protected	×	×	×	×	×	×
public	private	×	✓	✓	×	×	×
protected	private	×	✓	×	×	×	×
private	private	×	×	×	×	×	×

Precautions:

- Private members cannot be accessed in derived classes
- Correct inheritance type must be used based on accessibility requirements
- Display functions should only access allowed members

Conclusion:

The problem demonstrates multi-level inheritance and the effect of different access specifiers. Son and GrandSon classes accessed only allowed members of Father and the sum of accessible members was calculated correctly

Problem 2: Different Types of Inheritance

Objectives:

- To learn and test different types of inheritance in C++
- To access private, protected and public members from derived classes
- To understand visibility rules for each inheritance type

Theory:

Inheritance in C++ allows one class to derive properties and behaviors from another class. There are multiple types:

1. **Single Inheritance:** One base, one derived class
2. **Multi-Level Inheritance:** Chain of inheritance (base → derived → derived)
3. **Multiple Inheritance:** One derived class inherits from multiple base classes
4. **Hierarchical Inheritance:** Multiple derived classes inherit from the same base
5. **Hybrid Inheritance:** Combination of more than one type of inheritance

Access specifiers (private, protected, public) control which members are visible in derived classes

Codes:

1. Single Inheritance

```
Lab 3 > C++ 2.1.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  private:
6      int x;
7  protected:
8      int y;
9  public:
10     int z;
11     public:
12     A() { x=10; y=20; z=30; }
13 };
14
15 class B : public A {
16 public:
17     void show() {
18         // cout << x; // Not accessible
19         cout << "y: " << y << endl; // Accessible
20         cout << "z: " << z << endl; // Accessible
21     }
22 };
23
24 int main() {
25     B b;
26     b.show();
27     return 0;
28 }
```

Output:

```
y: 20
z: 30
```

2. Multi-Level Inheritance

```

Lab 3 > C++ 2.2.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  private:
6      int x;
7  protected:
8      int y;
9  public:
10     int z;
11  public:
12     A() { x=10; y=20; z=30; }
13 };
14
15 class B : public A {};
16
17 class C : public B {
18 public:
19     void show() {
20         // cout << x; // Not accessible
21         cout << "y: " << y << endl; // Accessible
22         cout << "z: " << z << endl; // Accessible
23     }
24 };
25
26 int main() {
27     C c;
28     c.show();
29     return 0;
30 }

```

Output:

```

y: 20
z: 30

```

3. Multiple Inheritance

```

Lab 3 > C++ 2.3.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  private:
6      int x;
7  protected:
8      int y;
9  public:
10     int z;
11  public:
12     A() { x=10; y=20; z=30; }
13 };
14
15 class B {
16 private:
17     int p;
18 protected:
19     int q;
20 public:
21     int r;
22 public:
23     B() { p=5; q=15; r=25; }
24 };
25
26 class C : public A, public B {
27 public:
28     void show() {
29         // cout << x; // Not accessible
30         cout << "y: " << y << endl; // Accessible
31         cout << "z: " << z << endl; // Accessible
32         // cout << p; // Not accessible
33         cout << "q: " << q << endl; // Accessible
34         cout << "r: " << r << endl; // Accessible
35     }
36 };
37
38 int main() {
39     C c;
40     c.show();
41     return 0;
42 }

```

Output:

```

y: 20
z: 30
q: 15
r: 25

```

4. Hierarchical Inheritance

```
Lab 3 > C++ 2.4.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  private:
6      int x;
7  protected:
8      int y;
9  public:
10     int z;
11     public:
12     A() { x=10; y=20; z=30; }
13 };
14
15 class B : public A {
16 public:
17     void show() {
18         // cout << x; // Not accessible
19         cout << "y: " << y << endl;
20         cout << "z: " << z << endl;
21     }
22 };
23
24 class C : public A {
25 public:
26     void show() {
27         // cout << x; // Not accessible
28         cout << "y: " << y << endl;
29         cout << "z: " << z << endl;
30     }
31 };
32
33 int main() {
34     B b;
35     C c;
36     b.show();
37     c.show();
38     return 0;
39 }
```

Output:

```
y: 20
z: 30
y: 20
z: 30
```

5. Hybrid Inheritance

```
Lab 3 > C++ 2.5.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  private:
6      int x;
7  protected:
8      int y;
9  public:
10     int z;
11     public:
12     A() { x=10; y=20; z=30; }
13 };
14
15 class B : public A {};
16 class C : public A {};
17
18 class D : public B, public C {
19 public:
20     void show() {
21         // cout << x; // Not accessible
22         cout << "B::y: " << B::y << endl;
23         cout << "B::z: " << B::z << endl;
24         cout << "C::y: " << C::y << endl;
25         cout << "C::z: " << C::z << endl;
26     }
27 };
28
29 int main() {
30     D d;
31     d.show();
32     return 0;
33 }
```

Output:

```
B::y: 20
B::z: 30
C::y: 20
C::z: 30
```

Procedure:

1. Classes were created according to the inheritance type
2. Derived class methods were written to access base class members
3. main() called the methods to observe accessible members
4. Each inheritance type was tested separately
5. **Output:** was noted for private, protected and public members

Observations:

- Private members of base classes were never accessible in derived classes
- Protected members were accessible in derived classes depending on inheritance type
- Public members were accessible in public inheritance
- Different inheritance types demonstrated different member accessibilities

Precautions

- Private members cannot be accessed directly in derived classes
- Correct inheritance type must be used for expected access
- Display functions should not try to access inaccessible members

Conclusion

The problem successfully demonstrated all five types of inheritance in C++. Access specifiers correctly controlled visibility in derived classes. Multi-level, multiple, hierarchical and hybrid inheritance types were successfully tested, showing how inheritance type and access specifiers influence accessibility

Problem 3: Constructors & Destructors in Inheritance

Objectives:

- To understand constructor and destructor execution in different types of inheritance.
- To implement constructors and destructors for single, multi-level, multiple, hierarchical and hybrid inheritance.
- To observe and verify the sequence of constructor and destructor calls.

Theory:

In C++, constructors are executed from base class to derived class, while destructors are executed from derived class to base class

1. Single Inheritance

```
Lab 3 > C++ 3.1.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  private:
6      int ax;
7  public:
8      A() { ax = 10; } // Constructor
9      ~A() { } // Destructor
10     int getAx() { return ax; }
11 };
12
13 class B : public A {
14 private:
15     int bx;
16 public:
17     B() { bx = 20; } // Constructor
18     ~B() { } // Destructor
19     void sum() { cout << getAx() + bx << endl; } // Sum method
20 };
21
22 int main() {
23     B b;
24     b.sum();
25     return 0;
26 }
```

Sequence:

Constructor: $A \rightarrow B$

Destructor: $B \rightarrow A$

2. Multi-Level Inheritance

```
Lab 3 > C++ 3.2.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  private:
6      int ax;
7  public:
8      A() { ax = 10; } // Constructor
9      ~A() { } // Destructor
10     int getAx() { return ax; }
11 };
12
13 class B : public A {
14 private:
15     int bx;
16 public:
17     B() { bx = 20; } // Constructor
18     ~B() { } // Destructor
19     int getBx() { return bx; }
20 };
21
22 class C : public B {
23 private:
24     int cx;
25 public:
26     C() { cx = 30; } // Constructor
27     ~C() { } // Destructor
28     void sum() { cout << getAx() + getBx() + cx << endl; } // Sum method
29 };
30
31 int main() {
32     C c;
33     c.sum();
34     return 0;
35 }
```

Sequence:

Constructor: $A \rightarrow B \rightarrow C$

Destructor: $C \rightarrow B \rightarrow A$

3. Multiple Inheritance

```

Lab 3 > C++ 3.3.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  private:
6      int ax;
7  public:
8      A() { ax = 10; } // Constructor
9      ~A() { } // Destructor
10     int getAx() { return ax; }
11 };
12
13 class B {
14 private:
15     int bx;
16 public:
17     B() { bx = 20; } // Constructor
18     ~B() { } // Destructor
19     int getBx() { return bx; }
20 };
21
22 class C : public A, public B {
23 private:
24     int cx;
25 public:
26     C() { cx = 30; } // Constructor
27     ~C() { } // Destructor
28     void sum() { cout << getAx() + getBx() + cx << endl; } // Sum method
29 };
30
31 int main() {
32     C c;
33     c.sum();
34     return 0;
35 }

```

Sequence:

Constructor: $A \rightarrow B \rightarrow C$

Destructor: $C \rightarrow B \rightarrow A$

4. Hierarchical Inheritance

```

Lab 3 > C++ 3.4.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  private:
6      int ax;
7  public:
8      A() { ax = 10; } // Constructor
9      ~A() { } // Destructor
10     int getAx() { return ax; }
11 };
12
13 class B : public A {
14 private:
15     int bx;
16 public:
17     B() { bx = 20; } // Constructor
18     ~B() { } // Destructor
19     void sum() { cout << getAx() + bx << endl; } // Sum method
20 };
21
22 class C : public A {
23 private:
24     int cx;
25 public:
26     C() { cx = 30; } // Constructor
27     ~C() { } // Destructor
28     void sum() { cout << getAx() + cx << endl; } // Sum method
29 };
30
31 int main() {
32     B b;
33     b.sum();
34     C c;
35     c.sum();
36     return 0;
37 }

```

Sequence:

For B: Constructor $A \rightarrow B$, Destructor $B \rightarrow A$

For C: Constructor $A \rightarrow C$, Destructor $C \rightarrow A$

5. Hybrid Inheritance

```
Lab 3 > C++ 3.5.cpp > ...
1 #include <iostream>
2 using namespace std;
```

```
3
4 class A {
5 private:
6     int ax;
7 public:
8     A() { ax = 10; } // Constructor
9     virtual ~A() { } // Destructor
10    int getAx() { return ax; }
11 };
12
13 class B : virtual public A {
14 private:
15     int bx;
16 public:
17     B() { bx = 20; } // Constructor
18     ~B() { } // Destructor
19     int getBx() { return bx; }
20 };
21
22 class C : virtual public A {
```

```
23 private:
24     int cx;
25 public:
26     C() { cx = 30; } // Constructor
27     ~C() { } // Destructor
28     int getCx() { return cx; }
29 };
30
31 class D : public B, public C {
32 private:
33     int dx;
34 public:
35     D() { dx = 40; } // Constructor
36     ~D() { } // Destructor
37     void sum() { cout << getAx() + getBx() + getCx() + dx << endl; } // Sum method
38 };
39
40 int main() {
41     D d;
42     d.sum();
43     return 0;
44 }
```

Sequence:

Constructor: $A \rightarrow B \rightarrow C \rightarrow D$

Destructor: $D \rightarrow C \rightarrow B \rightarrow A$

Procedure:

1. Classes were created with private members and constructors/destructors
2. Derived classes implemented their own constructors and destructors
3. Methods were written to calculate the sum of all relevant members
4. main() objects were created to observe constructor/destructor sequences
5. Execution order of constructors and destructors was noted for each inheritance type

Output:

- Constructors always executed from base $\rightarrow$ derived
- Destructors executed in reverse: derived $\rightarrow$ base
- Virtual inheritance ensured single construction of shared base class in diamond problem
- Sum methods returned correct results using accessible members

Precautions:

- Always initialize base members properly
- In multiple inheritance, constructors for each base must be correctly called
- For diamond inheritance, use virtual to prevent multiple base construction

Conclusion:

Constructors and destructors in inheritance follow strict sequences: base to derived for construction and derived to base for destruction. This problem demonstrated constructor/destructor execution clearly for single, multi-level, multiple, hierarchical and hybrid inheritance types

Problem 1: Method / Function Overloading

Objectives:

- To implement function overloading using a single method name
- To perform summation of different types and numbers of values

Theory:

Function overloading allows multiple functions to have the same name but different parameter lists. The compiler decides which function to call based on the number and type of arguments passed. This improves code readability and flexibility by using one logical function name for related operations

Code:

```
Lab 4 > C++ 1.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class Test {
5  public:
6      int Sum(int a) {
7          return a; // One integer
8      }
9      int Sum(int a, int b) {
10         return a + b; // Two integers
11     }
12     double Sum(double a, int b) {
13         return a + b; // Double and integer
14     }
15     double Sum(int a, double b) {
16         return a + b; // Integer and double
17     }
18     int Sum(double a, double b) {
19         return a + b; // Two double
20     }
21 };
23 int main() {
24     Test t;
25     // Calls to overloaded Sum() methods
26     cout << t.Sum(10) << endl;
27     cout << t.Sum(10,20) << endl;
28     cout << t.Sum(5.7,20) << endl;
29     cout << t.Sum(10,2.6) << endl;
30     cout << t.Sum(10.5,20.5) << endl;
31     return 0;
32 }
33 }
```

Procedure:

1. A Test class was created
2. The Sum() method was overloaded with different parameter types
3. Function calls were made from main()
4. Returned values were displayed

Output:

```
10
30
25.7
12.6
31
```

- Sum(10) → 10
- Sum(10,20) → 30
- Sum(5.7,20) → 25.7
- Sum(10,2.6) → 12.6
- Sum(10.5,20.5) → 31

Precautions:

- Parameter types must differ clearly
- Return type alone should not differ
- Proper data types must be used

Conclusion:

Function overloading was implemented successfully. The program correctly selected the appropriate Sum() method based on function arguments, demonstrating compile time polymorphism

Problem 2: Operator Overloading

Objectives:

- To calculate current in an AC circuit using operator overloading
- To represent complex numbers using a class

Theory:

In AC circuits, impedance and voltage are represented as complex numbers. Operator overloading allows arithmetic operations on user defined objects like complex numbers. This simplifies calculations and makes the code more intuitive and readable

Code:

```
Lab 4 > C++ 2.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class Circuit {
5  private:
6      float real;
7      float img;
8  public:
9      // Constructor to initialize real and imaginary
10     Circuit(float r = 0, float i = 0) {
11         real = r;
12         img = i;
13     }
14     // Operator overloading for addition
15     Circuit operator+(Circuit c) {
16         return Circuit(real + c.real, img + c.img);
17     }
18     // Operator overloading for division
19     Circuit operator/(Circuit c) {
20         float d = c.real*c.real + c.img*c.img;
21
22         return Circuit(
23             (real*c.real + img*c.img)/d,
24             (img*c.real - real*c.img)/d
25         );
26     }
27     // To display real and imaginary values
28     void display() {
29         cout << real << " + j" << img << endl;
30     }
31 };
32
33 int main() {
34     // To initialize impedance values
35     Circuit z1(3,4), z2(4,-3), z3(0,6);
36     // Input voltage
37     Circuit v(100,50);
38     // Current calculation
39     Circuit i = v / (z1 + z2 + z3);
40     i.display();
41     return 0;
42 }
```

Procedure:

1. The Circuit class was created to represent complex numbers
2. Operators were overloaded for addition and division
3. Impedances and voltage were initialized
4. Current was calculated and displayed

Output:

10.7143 + j-3.57143

- Complex arithmetic was performed correctly
- Current value was obtained using overloaded operators

Precautions:

- Division by zero must be avoided
- Correct formula for complex division should be used

Conclusion:

Operator overloading simplified complex number calculations. The circuit current was computed correctly, demonstrating effective use of operator overloading

Problem 3: Unary Operator Overloading

Objectives:

- To control TV sound using unary operator overloading.
- To increase and decrease sound using operators.

Theory:

Unary operator overloading allows operators like ++ and -- to be redefined for user-defined classes. This is useful for controlling incremental changes such as sound level in a TV remote system.

Code:

```
Lab 4 > C++ 3.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class Remote {
5  private:
6      int sound;
7
8  public:
9      Remote() {
10         sound = 0;
11     } // To initialize sound value
12
13     void operator++() {
14         sound++;
15     } // To increase sound
16
17     void operator--() {
18         sound--;
19     } // To decrease sound
20
21     void display() {
22         cout << sound << endl;
23     } // To display sound value
24 };
25
26 int main() {
27     Remote rs;
28     ++rs; // Increase
29     ++rs;
30     --rs; // Decrease
31     rs.display();
32     return 0;
33 }
```

Procedure:

1. A Remote class was created
2. Unary operators ++ and -- were overloaded
3. Sound level was increased and decreased
4. Final sound value was displayed

Output: 1

- Sound increased on ++ operation
- Sound decreased on -- operation
- Final sound level displayed correctly

Precautions:

- Sound should not exceed limits
- Initial sound value should be defined

Conclusion:

In this problem, Unary operator overloading was implemented successfully to control the sound levels of a TV remote. Each operator (++ and --) correctly modified the private sound data member as intended, simulating the increase and decrease of volume. The program demonstrated how unary operators can be customized for user-defined classes, and the final output verified that the remote responded accurately to multiple button presses. This exercise reinforced the concept of operator overloading and showed its practical application in modeling real-world devices

Problem 4: Friend Function**Objectives:**

- To calculate total money using friend function
- To access private data members of two classes

Theory:

Friend functions are special functions in C++ that are granted access to the private and protected members of one or more classes. This capability is particularly useful when operations require combining or manipulating data from multiple classes while still preserving the principles of encapsulation and data hiding, allowing controlled interaction between class internals without exposing them publicly

Code:

```
Lab 4 > C++ 4.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class Bank2;
5
6  class Bank1 {
7  private:
8      float money1;
9
10 public:
11     // Constructor to initialize money1
12     Bank1(float m) {
13         money1 = m;
14     }
15
16     // Friend function declaration
17     friend void Sum(Bank1, Bank2);
18 };
19
20 class Bank2 {
21 private:
22     float money2;
23
24 public:
25     // Constructor to initialize money2
26     Bank2(float m) {
27         money2 = m;
28     }
29
30     // Friend function declaration
31     friend void Sum(Bank1, Bank2);
32 };
33
34 // Friend function to calculate total money
35 void Sum(Bank1 b1, Bank2 b2) {
36     cout << b1.money1 + b2.money2 << endl;
37 }
38
39 int main() {
40     Bank1 b1(2000);
41     Bank2 b2(4000);
42
43     Sum(b1, b2);
44     return 0;
45 }
```

Procedure:

1. Two bank classes were created with private money variables
2. A friend function was declared and defined
3. Total money was calculated and displayed

Output: 6000

- Correct total money was displayed
- Private data was accessed successfully

Precautions:

- Friend functions should be used carefully
- Class relationships should be clear

Conclusion:

The friend function successfully accessed private data members from two classes. Total money calculation was performed correctly, demonstrating controlled data sharing

Problem 1: Method/Function Overriding

Objectives:

- To understand the concept of method/function overriding in C++ inheritance
- To observe the effect of the virtual keyword on runtime polymorphism

Theory:

In C++, a derived class can provide its own implementation of a base class method, which is called function overriding. When a base class pointer points to a derived class object, the behavior differs depending on whether the base class method is declared virtual. Using virtual enables runtime polymorphism, allowing the correct overridden method to be called.

Code:

```
Lab 5 > G+ 1.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  public:
6      void Print() {
7          cout << "Inside Print() of class A" << endl;
8      }
9  };
10
11 class B : public A {
12 public:
13     void Print() { // Overridden in derived class
14         cout << "Inside Print() of class B" << endl;
15     }
16 };
17
18 int main() {
19     // Part i
20     A a;
21     a.Print();
22
23     // Part ii
24     B b;
25     b.Print();
26
27     // Part iii
28     A* p1;
29     p1 = &a;
30     p1->Print();
31
32     // Part iv
33     A* p2;
34     p2 = &b;
35     p2->Print();
36
37     return 0;
38 }
```

Procedure:

1. Base class A was created with method Print()
2. Derived class B was created, and its Print() method was overridden
3. Objects of A and B were initialized, and their Print() methods were called (parts i and ii)
4. Base class pointers were assigned to A and B objects, and the Print() methods were called (parts iii and iv)
5. The output was observed without the virtual keyword to note which method was called.
6. The virtual keyword was added to Print() in class A
7. Steps 3-4 were repeated, and the output was observed for runtime polymorphism

Output:

```
Inside Print() of class A
Inside Print() of class B
Inside Print() of class A
Inside Print() of class A
```

Precautions:

- Correct inheritance syntax class B: public A was ensured
- Method signatures were matched exactly during overriding (same name, parameters, and return type)

Conclusion:

Function overriding allowed the derived class to provide its own implementation of the base class method. By adding the virtual keyword in the base class, runtime polymorphism was achieved, enabling the correct derived class method to execute when base class pointers were used. This demonstrated dynamic binding in C++ inheritance

Problem 2: Pure Virtual Function

Objectives:

- To understand the concept of pure virtual functions and abstract classes in C++
- To observe how derived classes implement abstract methods and enable runtime polymorphism

Theory:

A pure virtual function in C++ is declared by assigning 0 in its declaration (virtual void func() = 0). A class containing at least one pure virtual function becomes an abstract class, which cannot be instantiated. Derived classes must provide implementations for pure virtual functions to become concrete and usable objects

Code:

```
lab5 > G++ 2.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  public:
6      virtual void Print() = 0; // Pure virtual function
7  };
8
9  class B : public A {
10 public:
11     void Print() { // Implementation of pure virtual function
12         cout << "Inside Print() of class B" << endl;
13     }
14 };
15
16 int main() {
17     // Part iii (Invalid: cannot instantiate abstract class)
18     // A a;
19     // a.Print();
20
21     // Part iv
22     B b;
23     b.Print();
24
25     // Part iii using pointer (Invalid: cannot instantiate abstract class)
26     // A* p1;
27     // A a_obj;
28     // p1 = &a_obj;
29     // p1->Print();
30
31     // Part iv using pointer
32     A* p2;
33     p2 = &b;
34     p2->Print();
35
36     return 0;
37 }
```

Procedure:

1. Abstract class A was created with a pure virtual function Print()
2. Derived class B was created, and Print() was implemented in it
3. Objects of A were not instantiated since A was abstract

- Object of B was initialized, and its Print() method was called
- Base class pointers were assigned to B objects, and the Print() method was invoked to demonstrate runtime polymorphism
- Attempts to instantiate abstract class A were avoided to prevent compilation errors

Output:

```
Inside Print() of class B
Inside Print() of class B
```

Precautions:

- Abstract classes were not instantiated, as it is illegal in C++
- Pure virtual functions were correctly implemented in derived classes to avoid errors

Conclusion:

The use of pure virtual functions enforced that derived classes provide specific implementations for abstract methods. Runtime polymorphism was achieved using base class pointers pointing to derived class objects, demonstrating dynamic binding and abstract class behavior in C++

Problem 3: Exception Handling

Objectives:

- To understand the use of exception handling in C++ to manage runtime errors
- To implement multiple catch blocks to handle different types of exceptions

Theory:

Exception handling in C++ allows programs to manage runtime errors without crashing. Using try, throw, and catch blocks, specific exceptions can be detected and handled. Multiple catch blocks can handle different types of exceptions, and a default catch() block can catch all unhandled exceptions

Code:

```
Lab 5 > G++ 3.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int i;
7      int ax[5] = {10, 20, 60, 40, 30};
8
9      try
10     {
11         cout << "Enter index: ";
12         cin >> i;
13         if (i < 0 || i >= 5)
14             throw i; // First catch block for integer
15         cout << "ax[" << i << "] = " << ax[i] << endl;
16     }
17     catch (int e)
18     {
19         cout << "Exception caught: Invalid index " << e << endl;
20     }
21     catch (const char *msg)
22     {
23         cout << "Exception caught: " << msg << endl;
24     }
25     catch (...)
26     {
27         cout << "Unknown exception caught" << endl;
28     }
29     return 0;
30 }
```

Procedure:

- Array ax[] was initialized with five integer elements
- User input was requested to obtain the array index i

3. The try block was used to access ax[i]
4. Index validation was performed, and i was thrown as an exception if out of range
5. First catch block was written to receive the integer value of i
6. Second catch block was written to handle a string message "Out of Range Error"
7. Default catch() block was written to handle any other unhandled exceptions

Output:

```
Enter index: 1 Enter index: 5
ax[1] = 20      Exception caught: Invalid index 5
```

Precautions:

- Array bounds were checked before accessing elements to prevent runtime errors
- Appropriate exception types were thrown and caught to ensure program stability

Conclusion:

Exception handling successfully captured out-of-range index errors and prevented program termination. Multiple catch blocks allowed handling of different exception types, demonstrating robust runtime error management in C++

Problem 4: STL: Array

Objectives:

- To understand the use of STL array and its member functions in C++
- To perform various operations on an array object like access, fill, size, and address retrieval

Theory:

The STL array in C++ is a container that encapsulates fixed-size arrays with additional member functions. Functions like at(), front(), back(), fill(), empty(), size(), max_size(), begin(), and end() allow safe and convenient operations on arrays, enhancing functionality over traditional C-style arrays

Code:

```
Lab 5 > G++ 4.cpp > ...
1  #include <iostream>
2  #include <array>
3  using namespace std;
4
5  int main()
6  {
7      array<int, 6> ax;
8
9      // i) Assign values using single statement
10     ax = {10, 60, 30, 70, 20, 0};
11
12     // ii) Print third element using at()
13     cout << "Third element: " << ax.at(2) << endl;
14
15     // iii) Print first element using front()
16     cout << "First element: " << ax.front() << endl;
17
18     // iv) Print last element using back()
19     cout << "Last element: " << ax.back() << endl;
20
21     // v) Fill array elements with a specific value
22     ax.fill(5);
23     cout << "Array after fill: ";
24
25     for (int i = 0; i < ax.size(); i++)
26         cout << ax[i] << " ";
27     cout << endl;
28
29     // vi) Check whether array is empty
30     if (ax.empty())
31         cout << "Array is empty" << endl;
32     else
33         cout << "Array is not empty" << endl;
34
35     // vii) Print size of array
36     cout << "Size of array: " << ax.size() << endl;
37
38     // viii) Print maximum size of array
39     cout << "Maximum size: " << ax.max_size() << endl;
40
41     // ix) Print address of first element
42     cout << "Address of first element: " << ax.begin() << endl;
43
44     // x) Print address of last element
45     cout << "Address of last element: " << ax.end() << endl;
46
47     return 0;
}
```

Procedure:

1. STL array ax was declared with 6 elements
2. Values 10, 60, 30, 70, 20, 0 were assigned using a single statement
3. Third element was accessed and printed using at()
4. First element was accessed using front() and printed
5. Last element was accessed using back() and printed
6. All elements of the array were filled using fill() function
7. Array emptiness was tested using empty() function
8. Size of the array was printed using size()
9. Maximum possible size of the array was printed using max_size()
10. Address of the first element was printed using begin()
11. Address of the last element was printed using end()

Output:

```
Third element: 30
First element: 10
Last element: 0
Array after fill: 5 5 5 5 5 5
Array is not empty
Size of array: 6
Maximum size: 6
Address of first element: 0x61fef0
Address of last element: 0x61ff08
```

Precautions:

- Array indices were ensured to be within valid range to avoid runtime errors
- Correct STL array functions were used according to their syntax and functionality

Conclusion:

STL array provided a convenient and safe way to store and manipulate fixed-size elements. Member functions like at(), front(), back(), fill(), empty(), size(), max_size(), begin(), and end() were successfully demonstrated, showing the advantages of STL arrays over traditional arrays

Problem 5: STL: Pair Class

Objectives:

- To understand the use of STL pair class for storing two heterogeneous data elements
- To perform operations like assignment, access, modification, and swapping of pair objects

Theory:

The STL pair class allows two values of possibly different types to be stored together as a single object. Members first and second are used to access the elements. Functions like make_pair(), get<>(), and swap() allow assignment, modification, and exchange of pair objects efficiently

Code:

```
Lab 5 > G++ Scpp > ...
1 #include <iostream>
2 #include <utility>
3 using namespace std;
4
5 int main()
6 {
7     // i) Assign values using make_pair()
8     pair<int, string> px = make_pair(10, "Rajshahi");
9
10    // ii) Print int data member
11    cout << "First element (int): " << px.first << endl;
12
13    // iii) Print string data member
14    cout << "Second element (string): " << px.second << endl;
15
16    // iv) Modify first data member using get<>()
17    get<0>(px) = 20;
18    cout << "Modified first element: " << px.first << endl;
19
20    // v) Declare another pair and swap
21    pair<int, string> bx = make_pair(50, "Dhaka");
22    px.swap(bx);
23
24    cout << "After swap, px: (" << px.first << ", " << px.second << ")" << endl;
25    cout << "After swap, bx: (" << bx.first << ", " << bx.second << ")" << endl;
26
27    return 0;
28 }
```

Procedure:

1. Pair object px was declared with int and string types
2. Values 10 and "Rajshahi" were assigned using make_pair()
3. Integer member was accessed and printed using first
4. String member was accessed and printed using second
5. First data member was modified to 20 using get<>()
6. Another pair object bx was declared and initialized with values
7. Pair objects px and bx were swapped using swap() function
8. The results after swapping were printed to verify correctness

Output:

```
First element (int): 10
Second element (string): Rajshahi
Modified first element: 20
After swap, px: (50, Dhaka)
After swap, bx: (20, Rajshahi)
```

Precautions:

- Correct data types were ensured for both elements while declaring the pair
- Proper STL functions (make_pair(), get<>(), swap()) were used to avoid syntax or runtime errors

Conclusion:

STL pair provided a convenient way to store two heterogeneous elements together. Assignment, access, modification, and swapping of pairs were successfully demonstrated, showing the efficiency and versatility of the pair class in C++

Problem 6: STL: Tuple Class

Objectives:

- To understand the use of STL tuple class for storing multiple heterogeneous data elements
- To perform operations like assignment, access, modification, and swapping of tuple objects

Theory:

The STL tuple class allows storing multiple elements of different types in a single object. Elements are accessed and modified using the `get<>()` function. Tuples can also be swapped with other tuples using the `swap()` function, making them flexible for managing heterogeneous data together

Code:

```
Lab 5 > G++ 8.cpp >...
1 #include <iostream>
2 #include <tuple>
3 using namespace std;
4
5 int main()
6 {
7     // i) Assign values using make_tuple()
8     tuple<int, string, double> tx = make_tuple(100, "Fardin", 3.5);
9
10    // ii) Print int data member
11    cout << "Integer element: " << get<0>(tx) << endl;
12
13    // iii) Print string data member
14    cout << "String element: " << get<1>(tx) << endl;
15
16    // iv) Print double data member
17    cout << "Double element: " << get<2>(tx) << endl;
18
19    // v) Modify third data member to 3.7
20    get<2>(tx) = 3.7;
21    cout << "Modified double element: " << get<2>(tx) << endl;
22
23    // vi) Declare another tuple and swap
24    tuple<int, string, double> bx = make_tuple(200, "Numan", 4.2);
25    tx.swap(bx);
26
27    cout << "After swap, tx: (" << get<0>(tx) << ", " << get<1>(tx) << ", " << get<2>(tx) << ")" << endl;
28    cout << "After swap, bx: (" << get<0>(bx) << ", " << get<1>(bx) << ", " << get<2>(bx) << ")" << endl;
29
30    return 0;
31 }
```

Procedure:

1. Tuple object tx was declared with int, string, and double types
2. Values <100, "Kamal", 3.5> were assigned using `make_tuple()`
3. First element was accessed and printed using `get<0>()`
4. Second element was accessed and printed using `get<1>()`
5. Third element was accessed and printed using `get<2>()`
6. Third element was modified to 3.7 using `get<2>()`
7. Another tuple object bx was declared and initialized with values
8. Tuples tx and bx were swapped using `swap()` function
9. The results after swapping were printed to verify correctness

Output:

```
Integer element: 100
String element: Fardin
Double element: 3.5
Modified double element: 3.7
After swap, tx: (200, Numan, 4.2)
After swap, bx: (100, Fardin, 3.7)
```

Precautions:

- Correct data types were ensured while declaring the tuple to avoid type mismatch
- `get<>()` function was used with the correct index to access or modify elements

Conclusion:

STL tuple allowed storing multiple heterogeneous elements in a single object. Assignment, access, modification, and swapping were successfully demonstrated, showing the efficiency and flexibility of tuples in C++

Problem 1: STL: Stack Class

Objectives:

- To understand the use of STL stack class for managing data in LIFO (Last In, First Out) order
- To implement stack operations like push, pop, top, and empty using menu-driven programming

Theory:

The STL stack class in C++ provides a container adapter to store elements in LIFO order. Functions like push(), pop(), top(), and empty() allow efficient insertion, deletion, access, and status checking of the stack. Menu-driven programs enable dynamic user interaction with stack operations

Code:

```
Lab 6 > G++ 1.cpp > ...
1  #include <iostream>
2  #include <stack>
3  using namespace std;
4
5  int main(){
6      stack<int> s;
7      int choice, value;
8
9      do{
10         cout << "\n** Stack **\n";
11         cout << "1. Push\n";
12         cout << "2. Pop\n";
13         cout << "3. Display top element\n";
14         cout << "4. Exit\n";
15         cout << "Enter your option: ";
16         cin >> choice;
17
18         switch (choice){
19             case 1: // Push
20                 cout << "Enter value to push: ";
21                 cin >> value;
22                 s.push(value);
23                 break;
24
25             case 2: // Pop
26                 if (s.empty()){
27                     cout << "Popped: " << s.top() << endl;
28                     s.pop();
29                 }
30                 else
31                     cout << "Stack is empty. Cannot pop.\n";
32                 break;
33
34             case 3: // Display top
35                 if (s.empty()){
36                     cout << "Top element: " << s.top() << endl;
37                 }
38                 else
39                     cout << "Stack is empty.\n";
40                 break;
41
42             case 4: // Exit
43                 cout << "Exiting program.\n";
44                 break;
45
46             default:
47                 cout << "Invalid option! Try again.\n";
48         } while (choice != 4);
49
50         return 0;
51     }
```

Procedure:

1. Stack object s was declared using STL stack class.
2. A menu-driven loop was created to take user input repeatedly until exit
3. User was prompted to push a value, which was added using push()
4. pop() was used to remove the top element when requested, after checking that the stack was not empty
5. top() was used to display the top element, provided the stack was not empty
6. empty() function was used to check whether the stack had any elements
7. Loop continued until the user selected the exit option

Output:

```
** Stack **
1. Push
2. Pop
3. Display top element
4. Exit
Enter your option: 1
Enter value to push: 10

** Stack **
1. Push
2. Pop
3. Display top element
4. Exit
Enter your option: 3
Top element: 10
```

Precautions:

- Stack underflow was prevented by checking empty() before performing pop() or top()
- Valid menu options were ensured to prevent incorrect program behavior

Conclusion:

STL stack provided an efficient way to manage data in LIFO order. Menu-driven programming allowed dynamic interaction, and functions like push(), pop(), top(), and empty() were successfully demonstrated, ensuring correct stack manipulation and error handling.

Problem 2: STL: Queue Class

Objectives:

- To understand the use of STL queue class for managing data in FIFO (First In, First Out) order
- To implement queue operations like enqueue, dequeue, front, back, and empty using menu-driven programming

Theory:

The STL queue class in C++ is a container adapter that stores elements in FIFO order. Functions like push(), pop(), front(), back(), and empty() allow efficient insertion, deletion, access to front and rear elements, and status checking of the queue. Menu-driven programs facilitate dynamic interaction with queue operations

Code:

```
Lab 6 > G++ 2.cpp > ...
1  #include <iostream>
2  #include <queue>
3  using namespace std;
4
5  int main(){
6      queue<int> q;
7      int choice, value;
8
9      do{
10         cout << "\n** Queue **\n";
11         cout << "1. Enqueue\n";
12         cout << "2. Dequeue\n";
13         cout << "3. Display Front\n";
14         cout << "4. Display Rear\n";
15         cout << "5. Exit\n";
16         cout << "Enter your option: ";
17         cin >> choice;
18
19         switch (choice){
20             case 1: // Enqueue
21                 cout << "Enter value to enqueue: ";
22                 cin >> value;
23                 q.push(value);
24                 break;
25             case 2: // Dequeue
26                 if (!q.empty()){
27                     cout << "Dequeued: " << q.front() << endl;
28                     q.pop();
29                 }
30
31             else{
32                 cout << "Queue is empty. Cannot dequeue.\n";
33             }
34             break;
35
36             case 3: // Display Front
37                 if (!q.empty()){
38                     cout << "Front element: " << q.front() << endl;
39                 }
40                 else{
41                     cout << "Queue is empty.\n";
42                 }
43                 break;
44
45             case 4: // Display Rear
46                 if (!q.empty()){
47                     cout << "Rear element: " << q.back() << endl;
48                 }
49                 else{
50                     cout << "Queue is empty.\n";
51                 }
52                 break;
53
54             case 5: // Exit
55                 cout << "Exiting program.\n";
56                 break;
57
58             default:
59                 cout << "Invalid option! Try again.\n";
60         }
61     } while (choice != 5);
62     return 0;
}
```

Procedure:

1. Queue object q was declared using STL queue class
2. A menu-driven loop was created to take user input repeatedly until exit
3. User input was taken to enqueue a value, which was added using push()

4. pop() was used to remove the front element after verifying that the queue was not empty
5. front() was used to display the front element of the queue, provided it was not empty
6. back() was used to display the rear element of the queue
7. empty() function was used to check whether the queue contained any elements
8. The menu loop continued until the user selected the exit option

Output:

```

** Queue **
1. Enqueue
2. Dequeue
3. Display Front
4. Display Rear
5. Exit
Enter your option: 1
Enter value to enqueue: 2403179

** Queue **
1. Enqueue
2. Dequeue
3. Display Front
4. Display Rear
5. Exit
Enter your option: 3
Front element: 2403179

```

Precautions:

- Queue underflow was prevented by checking empty() before performing pop(), front(), or back()
- Valid menu options were ensured to avoid incorrect program execution

Conclusion:

STL queue allowed efficient management of data in FIFO order. Menu-driven programming enabled dynamic interaction with the queue, and functions like push(), pop(), front(), back(), and empty() were successfully demonstrated, ensuring correct queue operations and error handling

Problem 3: STL: Vector Class

Objectives:

- To understand the use of STL vector class to simulate linked list operations in C++
- To implement vector operations like insertion, deletion, access, and modification using a menu-driven program

Theory:

The STL vector class in C++ is a dynamic array that can grow or shrink at runtime. Functions like push_back(), insert(), erase(), begin(), end(), pop_back(), and pop_front() allow flexible management of elements. Vectors can simulate linked list behavior for insertion, deletion, updating, and searching operations efficiently

Code:

```

Lab 6 > G++ 3.cpp > ...
1  #include <iostream>
2  #include <vector>
3  #include <algorithm> // for find()
4  using namespace std;
5
6  int main() {
7      vector<int> v;
8      int choice, value, pos;
9
10     do {
11         cout << "\n** Linked List **\n";
12         cout << "1. Insert\n";
13         cout << "2. Delete\n";
14         cout << "3. Update\n";
15         cout << "4. Search\n";
16         cout << "5. Exit\n";
17         cout << "Enter your option: ";
18         cin >> choice;
19
20         switch(choice) {
21             case 1: // Insert
22                 cout << "Enter value to insert: ";
23                 cin >> value;
24                 cout << "Enter position (0-based index, use -1 to insert at end): ";
25                 cin >> pos;
26                 if(pos == -1 || pos == v.size())
27                     v.push_back(value);
28                 else if(pos >= 0 && pos < v.size())
29                     v.insert(v.begin() + pos, value);
30                 else
31                     cout << "Invalid position!\n";
32                 break;
33
34             case 2: // Delete
35                 cout << "Enter position to delete: ";
36                 cin >> pos;
37                 if(pos >= 0 && pos < v.size())
38                     v.erase(v.begin() + pos);
39                 else
40                     cout << "Invalid position!\n";
41                 break;
42
43             case 3: // Update
44                 cout << "Enter position to update: ";
45                 cin >> pos;
46                 if(pos >= 0 && pos < v.size()) {
47                     cout << "Enter new value: ";
48                     cin >> value;
49                     v[pos] = value;
50                 } else {
51                     cout << "Invalid position!\n";
52                 }
53                 break;
54
55             case 4: // Search
56                 cout << "Enter value to search: ";
57                 cin >> value;
58                 {
59                     auto it = find(v.begin(), v.end(), value);
60                     if(it != v.end())
61                         cout << "Value found at position " << distance(v.begin(), it) << endl;
62                     else
63                         cout << "Value not found!\n";
64                 }
65                 break;
66
67             case 5:
68                 cout << "Exiting program.\n";
69                 break;
70
71             default:
72                 cout << "Invalid option! Try again.\n";
73         }
74
75         // Display current vector
76         cout << "Current List: ";
77         for(int x : v) cout << x << " ";
78         cout << endl;
79     } while(choice != 5);
80
81     return 0;
82 }

```

Procedure:

1. Vector object v was declared to store integer elements dynamically.
2. A menu-driven loop was created to allow multiple operations until exit.
3. insert() was used to add elements at user-specified positions.
4. erase() was used to delete elements at specified positions.
5. Elements were updated directly by accessing them using the index.
6. find() along with begin() and end() was used to search for a value in the vector.
7. push_back() was implicitly used if inserting at the end of the vector.
8. The current state of the vector was displayed after each operation to show changes.

Output:

```

** Linked List **
1. Insert
2. Delete
3. Update
4. Search
5. Exit
Enter your option: 1
Enter value to insert: 10
Enter position (0-based index, use -1 to insert at end): 0
Current List: 10

```

Precautions:

- Valid positions were ensured before performing insert(), erase(), or update() to avoid runtime errors
- Vector functions were used correctly according to their syntax to prevent invalid operations

Conclusion:

STL vector successfully simulated linked list operations like insertion, deletion, updating, and searching. Menu-driven programming allowed dynamic interaction, and vector functions such as insert(), erase(), begin(), and end() were demonstrated effectively for element management

Problem 1: UML

Objectives:

- To understand the structure of a class and its members using UML (Unified Modeling Language)
- To represent data members and member functions of a class visually

Theory:

UML (Unified Modeling Language) is a standardized way to visualize the design of a system. For classes, UML diagrams show data members (private, protected, public) and member functions along with their access specifiers, making it easier to understand the structure and relationships of a class.

Code:

```
Lab 7 > G++ 1.cpp > ...
1  class Test
2  {
3  private:
4      int x;
5
6  protected:
7      int y;
8
9  public:
10     int z;
11
12     public:
13     void SetData(int a, int b, int c)
14     {
15         x = a;
16         y = b;
17         z = c;
18     }
19
20     float getAverage()
21     {
22         return (x + y + z) / 3.0;
23     }
24 };
```

Procedure:

1. The class Test was analyzed to identify data members (x, y, z) and member functions (SetData(), getAverage())
2. Access specifiers (private, protected, public) of each member were noted
3. A UML diagram was drawn with three compartments:
 - a) Top compartment for the class name
 - b) Middle compartment for data members with access symbols (- for private, # for protected, + for public)
 - c) Bottom compartment for member functions with access symbols and parameter lists
4. Relationships, if any, would have been represented (not applicable in this single-class example)

Precautions:

- Correct access specifiers were ensured while labeling the data members and functions
- Function names and parameter types were correctly written in the UML diagram to avoid ambiguity

Conclusion:

The UML diagram effectively represented the Test class structure, showing private, protected, and public members along with their respective functions. This diagram provided a clear and visual understanding of the class design and encapsulation in C++.

Problem 2: UML: Aggregation

Objectives:

- To understand the concept of aggregation in object-oriented programming
- To implement a relationship where one class contains a reference to another class without owning its lifetime

Theory:

Aggregation represents a “has-a” relationship between two classes, where one class contains a reference (pointer or object) of another class but does not manage its lifetime. If the contained object (e.g., Patient) is destroyed, the container object (e.g., Doctor) still exists, demonstrating weak ownership. UML diagrams use a hollow diamond to show aggregation.

Code:

```
Lab 7 > G++ 2.cpp > ...
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Patient
6  {
7      string pname;
8      int age;
9
10 public:
11     Patient(string n, int a) : pname(n), age(a) {}
12     void display()
13     {
14         cout << "Patient Name: " << pname << ", Age: " << age << endl;
15     }
16 };
17
18 class Doctor
19 {
20     string dname;
21     int experience;
22
23 public:
24     Doctor(string n, int exp) : dname(n), experience(exp) {}
25
26     void show()
27     {
28         cout << "Doctor Name: " << dname << ", Experience: " << experience << " years" << endl;
29     }
30
31     void treat(Patient &p)
32     { // Aggregation: reference to patient
33         cout << dname << " is treating ";
34         p.display();
35     }
36
37 int main()
38 {
39     Doctor d1("Dr. Shyla", 15);
40     Patient p1("Fardin", 20);
41     Patient p2("Numan", 21);
42
43     d1.show();
44     d1.treat(p1);
45     d1.treat(p2);
46
47     return 0;
48 }
```

Procedure:

1. Class Patient was created to store patient information (name and age)
2. Class Doctor was created to store doctor information (name and experience)
3. A method treat() in Doctor was created to accept a reference to a Patient object, establishing aggregation
4. Doctor object was initialized independently of Patient objects
5. Patient objects were created and passed to the Doctor object's treat() method to demonstrate association
6. Destruction of Patient objects would not affect the Doctor object, showing aggregation

Output:

```
Dr. Shyla is treating Patient Name: Fardin, Age: 20
Dr. Shyla is treating Patient Name: Numan, Age: 21
```

Precautions:

- References or pointers were used to establish aggregation instead of creating objects inside the container class
- Proper initialization of objects was ensured to avoid dangling references or null pointer access

Conclusion:

Aggregation successfully demonstrated a “has-a” relationship where the Doctor class could interact with multiple Patient objects without owning them. This shows that contained objects can be destroyed independently while the container object continues to exist, as visualized in the UML diagram.

Problem 3: UML: Composition

Objectives:

- To understand the concept of composition in object-oriented programming
- To implement a relationship where one class owns another class entirely, and the lifetime of the contained object depends on the container class

Theory:

Composition represents a strong “has-a” relationship where one class owns another class. If the container object (e.g., Doctor) is destroyed, all contained objects (e.g., Patient) are automatically destroyed. UML diagrams use a filled diamond to show composition

Code:

```
Lab 7 > G++ 3.cpp > ...
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Patient
6  {
7      string pname;
8      int age;
9
10 public:
11     Patient(string n, int a) : pname(n), age(a) {}
12     void display()
13     {
14         cout << "Patient Name: " << pname << ", Age: " << age << endl;
15     }
16 };
17
18 class Doctor{
19     string dname;
20     int experience;
21     Patient p; // Composition: Patient is a member object of Doctor
22 public:
23     Doctor(string dn, int exp, string pn, int pa) : dname(dn), experience(exp), p(pn, pa) {}
24     void show()
25     {
26         cout << "Doctor Name: " << dname << ", Experience: " << experience << " years" << endl;
27     }
28     void treat()
29     {
30         cout << dname << " is treating ";
31         p.display();
32     }
33 };
34
35 int main()
36 {
37     Doctor d1("Dr. Shyla", 15, "Human", 20);
38     d1.show();
39     d1.treat();
40
41     return 0;
42 }
```

Procedure:

1. Class Patient was created to store patient information (name and age)
2. Class Doctor was created to store doctor information (name and experience)
3. Patient object was declared as a member variable of Doctor to establish composition
4. Doctor object was initialized, which automatically initialized the Patient object using the constructor
5. Method treat() in Doctor was called to display information of the contained Patient object
6. Upon destruction of the Doctor object, the Patient object was automatically destroyed, demonstrating strong ownership

Output:

```
Doctor Name: Dr. Shyla, Experience: 15 years  
Dr. Shyla is treating Patient Name: Numan, Age: 20
```

Precautions:

- Proper constructor initialization of member objects was ensured to avoid uninitialized objects
- Composition was established by declaring the member object inside the class, not using references or pointers

Conclusion:

Composition successfully demonstrated a “owns-a” relationship where the Patient object existed only as part of the Doctor object. Destroying the Doctor object also destroyed the Patient object, confirming strong ownership and proper resource management, as represented in the UML diagram

Problem 4: Java Programming - Basics

Objectives:

- To practice basic Java programming concepts including input/output, conditionals, arrays, and loops
- To perform array operations like finding largest, smallest, average, searching, and sorting

Theory:

Java is an object-oriented programming language used for building portable, platform-independent applications. Basic programming tasks involve reading inputs, performing computations, using arrays to store data, and manipulating elements with loops and methods like sorting or searching

Code:

```
Lab 7 > java4.java > ...
1  import java.util.Scanner;
2  import java.util.Arrays;
3
4  public class java4 {
5      public static void main(String[] args) {
6          Scanner sc = new Scanner(System.in);
7
8          // i) Display name and address
9          System.out.println("Name: Fardin Numan");
10         System.out.println("Address: Rajshahi, Bangladesh");
11
12         // ii) Take two integers and print the bigger one
13         System.out.print("Enter first number: ");
14         int a = sc.nextInt();
15         System.out.print("Enter second number: ");
16         int b = sc.nextInt();
17         if(a > b) System.out.println("Bigger number is: " + a);
18         else System.out.println("Bigger number is: " + b);
19
20         // iii) Array operations
21         int[] ax = new int[10];
22         System.out.print("Enter 10 numbers:");
23         for(int i=0; i<10; i++) {
24             ax[i] = sc.nextInt();
25         }
26
27         // Finding largest, smallest and average
28         int max = ax[0], min = ax[0], sum = 0;
29         for(int i=0; i<10; i++) {
30             if(ax[i] > max) max = ax[i];
31             if(ax[i] < min) min = ax[i];
32             sum += ax[i];
33         }
34         double avg = sum / 10.0;
35         System.out.println("Largest: " + max);
36         System.out.println("Smallest: " + min);
37         System.out.println("Average: " + avg);
38     }
39
40     // Searching for a number
41     System.out.print("Enter a number to search: ");
42     int key = sc.nextInt();
43     boolean found = false;
44     for(int i=0; i<10; i++) {
45         if(ax[i] == key) {
46             found = true;
47             break;
48         }
49     }
50     if(found) System.out.println(key + " is found in the array.");
51     else System.out.println(key + " is not found in the array.");
52
53     // Sorting array
54     Arrays.sort(ax);
55     System.out.print("Sorted array:");
56     for(int i=0; i<10; i++) {
57         System.out.print(ax[i] + " ");
58     }
59     System.out.println();
60     sc.close();
61 }
62 }
```

Procedure:

1. Scanner class was used to take input from the user
2. Name and address were printed using System.out.println()
3. Two integers were read from the user and compared using an if-else statement to find the bigger one
4. Array ax[] was declared to store 10 numbers, which were read from the user
5. The largest and smallest numbers were determined by iterating through the array
6. Sum of all elements was calculated to find the average
7. Linear search was implemented to find a specific number in the array
8. Arrays.sort() method was used to sort the array in ascending order
9. Array elements were displayed after sorting

Output:

Name: Fardin Numan	Enter 10 numbers:	Enter a number to search: 7
Address: Rajshahi, Bangladesh	1 2 3 4 5 6 7 8 9 10	7 is found in the array.
Enter first number: 7	Largest: 10	Sorted array:
Enter second number: 9	Smallest: 1	1 2 3 4 5 6 7 8 9 10
Bigger number is: 9	Average: 5.5	

Precautions:

- Array indices were carefully used to avoid ArrayIndexOutOfBoundsException
- Inputs were validated for integer types to avoid runtime errors

Conclusion:

Basic Java programming concepts were successfully applied to display information, compare numbers, and perform array operations including finding largest, smallest, average, searching, and sorting. The program demonstrated fundamental control structures and array handling in Java.

Problem 5: Java Programming - Private Method

Objectives:

- To understand the concept of private methods in Java and their access restrictions
- To demonstrate that private methods can be invoked only inside the class they are defined in

Theory:

In Java, a private method is accessible only within the class in which it is declared. It cannot be called from outside the class or by derived classes. This encapsulation ensures data hiding and controlled access to internal functionalities of a class

Code:

```
Lab 7 > java5.java > ...
1  class TestPrivate {
2      private void displayMessage() {
3          System.out.println("This is a private method.");
4      }
5
6      public void callPrivateMethod() {
7          // Private method can be called within the class
8          displayMessage();
9      }
10 }
11
12 public class java5 {
13     public static void main(String[] args) {
14         TestPrivate obj = new TestPrivate();
15
16         // Calling private method from outside the class
17         // obj.displayMessage(); // Error: displayMessage() has private access
18
19         // Calling private method indirectly using public method
20         obj.callPrivateMethod(); // Works fine
21     }
22 }
```

Procedure:

1. Class TestPrivate was created with a private method displayMessage()
2. A public method callPrivateMethod() was written to call the private method from inside the class
3. In the main() method, an object of TestPrivate was created
4. Attempt to call displayMessage() directly from main() was noted to be invalid (commented to prevent compilation error)
5. Private method was successfully invoked indirectly through a public method
6. Observations were made to verify that private methods cannot be accessed from outside the class

Output:

```
This is a private method.
```

Precautions:

- Private methods were only called inside the class to prevent compilation errors
- Correct access modifiers were used to demonstrate encapsulation clearly

Conclusion:

Private methods were successfully shown to be accessible only within the class and not from outside. Indirect access through public methods allowed controlled execution, demonstrating data hiding and encapsulation in Java

Problem 6: Java Programming - Static Method/Member

Objectives:

- To understand the behavior of static methods and static members in Java
- To demonstrate that static methods can access only static members and cannot directly access non-static members

Theory:

In Java, static members (variables or methods) belong to the class rather than individual objects. Static methods can directly access other static members of the class but cannot access non-static (instance) members unless an object of the class is created. This ensures shared data and behavior across all objects

Code:

```
Lab 7 > java6.java > ...
1  class TestStatic {
2      static int staticVar = 100; // Static variable
3      int nonStaticVar = 50; // Non-static variable
4
5      static void staticMethod() {
6          System.out.println("Inside static method.");
7
8          // Accessing static variable - Allowed
9          System.out.println("Static variable: " + staticVar);
10
11         // Accessing non-static variable - Not allowed directly
12         // System.out.println("Non-static variable: " + nonStaticVar);
13
14         // Accessing non-static variable using object
15         TestStatic obj = new TestStatic();
16         System.out.println("Non-static variable via object: " + obj.nonStaticVar);
17     }
18
19     void nonStaticMethod() {
20         System.out.println("Inside non-static method.");
21         System.out.println("Access static variable: " + staticVar); // Allowed
22         System.out.println("Access non-static variable: " + nonStaticVar); // Allowed
23     }
24 }
25
26 public class java6 {
27     public static void main(String[] args) {
28         // Call static method without creating object
29         TestStatic.staticMethod();
30
31         // Call non-static method using object
32         TestStatic obj = new TestStatic();
33         obj.nonStaticMethod();
34     }
35 }
```

Procedure:

1. Class TestStatic was created with a static variable staticVar and a non-static variable nonStaticVar

2. A static method `staticMethod()` was written to access the static variable directly and the non-static variable through an object
3. A non-static method `nonStaticMethod()` was written to access both static and non-static variables directly
4. In `main()`, the static method was called without creating an object, demonstrating class-level access
5. Non-static method was called using an object of the class, showing instance-level access
6. Observations confirmed that static methods cannot access non-static members directly

Output:

```
Inside static method.  
Static variable: 100  
Non-static variable via object: 50  
Inside non-static method.  
Access static variable: 100  
Access non-static variable: 50
```

Precautions:

- Non-static members were accessed through objects to avoid compilation errors in static methods
- Static methods were invoked using the class name to demonstrate correct usage

Conclusion:

Static methods were shown to access static members directly and non-static members only through objects, confirming Java's class-level behavior. This demonstrates proper use of static methods and members and the distinction between class-level and instance-level access

Problem 1: ArrayList Class

Objectives:

- To understand the use of ArrayList class for dynamic array management
- To implement common ArrayList methods like add(), get(), set(), remove(), size(), toString(), sort(), and clear().

Theory:

The ArrayList class in Java is a resizable array that allows dynamic storage of elements. It provides methods to add, access, modify, remove, sort, and clear elements efficiently. Unlike regular arrays, ArrayLists can grow or shrink at runtime, making them suitable for dynamic data management

Code:

```
Java1.java > ...
1  import java.util.ArrayList;
2  import java.util.Collections;
3
4  public class Java1 {
5      public static void main(String[] args) {
6
7          ArrayList<Integer> list = new ArrayList<>();
8
9          // i) add()
10         list.add(10);
11         list.add(30);
12         list.add(20);
13         System.out.println("After add(): " + list);
14
15         // ii) get()
16         System.out.println("Element at index 1 (get): " + list.get(index 1));
17
18         // iii) set()
19         list.set(index 1, element 50);
20         System.out.println("After set(): " + list);
21
22         // iv) remove()
23         list.remove(index 0);
24         System.out.println("After remove(): " + list);
25
26         // v) size()
27         System.out.println("Size of list: " + list.size());
28
29         // vi) toString()
30         System.out.println("List (toString): " + list.toString());
31
32         // vii) sort()
33         Collections.sort(list);
34         System.out.println("After sort(): " + list);
35
36         // viii) clear()
37         list.clear();
38         System.out.println("After clear(): " + list);
39     }
40 }
```

Procedure:

1. ArrayList object list was declared to store Integer elements dynamically
2. Elements were added using add() method
3. Element at index 1 was accessed using get() method
4. Element at index 1 was modified using set() method
5. First element was removed using remove() method
6. size() method was used to determine the number of elements
7. toString() method was used to display the ArrayList contents
8. ArrayList was sorted using Collections.sort()
9. All elements were cleared using clear() method

Output:

```
After add(): [10, 30, 20]
Element at index 1 (get): 30
After set(): [10, 50, 20]
After remove(): [50, 20]
Size of list: 2
List (toString): [50, 20]
After sort(): [20, 50]
After clear(): []
```

Precautions:

- Valid indices were ensured before using get(), set(), or remove() to prevent IndexOutOfBoundsException
- ArrayList methods were used correctly according to Java syntax to prevent runtime errors

Conclusion:

Java ArrayList provided dynamic storage and flexible management of integer elements. Methods like add(), get(), set(), remove(), size(), toString(), sort(), and clear() were successfully demonstrated, highlighting the convenience and efficiency of ArrayList over traditional arrays

Problem 2: Anonymous Object (MidPoint Method)

Objectives:

- To understand the use of anonymous objects in Java.
- To implement a method that calculates the midpoint of two Point objects.

Theory:

In Java, anonymous objects are objects created without being assigned to a reference variable. They are useful for one-time use. By passing objects directly to a method and returning a new object, operations like calculating the midpoint of two points can be performed efficiently without storing intermediate objects

Code:

```
Java2.java > ...
1 public class Java2 {
2
3     static class Point {
4         int x, y;
5
6         Point(int x, int y) {
7             this.x = x;
8             this.y = y;
9         }
10    }
11
12    public static Point MidPoint(Point p1, Point p2) {
13        return new Point(
14            (p1.x + p2.x) / 2,
15            (p1.y + p2.y) / 2);
16    }
17
18    Run | Debug
19    public static void main(String[] args) {
20
21        Point p1 = new Point(x: 2, y: 4);
22        Point p2 = new Point(x: 6, y: 8);
23
24        Point mid = MidPoint(p1, p2);
25
26        System.out.println("Midpoint: (" + mid.x + ", " + mid.y + ")");
27    }
}
```

Procedure:

1. Class Point was created with integer members x and y to represent coordinates
2. Two Point objects p1 and p2 were initialized with given coordinates
3. Method MidPoint() was defined to accept two Point objects and calculate their midpoint
4. A new anonymous Point object was returned by MidPoint() containing the midpoint coordinates
5. The returned midpoint object was stored in mid and its coordinates were displayed

Output:

Midpoint: (4, 6)

Precautions:

- Valid Point objects were ensured to avoid NullPointerException
- Arithmetic operations were performed carefully to prevent incorrect midpoint calculation

Conclusion:

Anonymous objects were effectively used to calculate and return the midpoint of two points. The method MidPoint() demonstrated how objects can be passed and returned without explicitly storing intermediate objects, improving code clarity and efficiency

Problem 3: Inner Class

Objectives:

- To understand the concept of inner classes in Java
- To implement a method in the inner class that accesses private members of both inner and outer classes

Theory:

An inner class in Java is a class defined within another class. Inner classes can access the private members of the outer class directly. This allows encapsulation of functionality and simplifies operations like summing private data members of outer and inner classes without exposing them

Code:

```
Java3.java > ...
1 public class Java3 {
2
3     static class A {
4         private int a = 10; // private member of A
5
6         class B {
7             private int b = 20; // private member of B
8
9             void sum() {
10                 System.out.println("Sum: " + (a + b));
11             }
12         }
13     }
14
15     public static void main(String[] args) {
16
17         A a = new A();
18         A.B b = a.new B();
19         b.sum();
20
21     }
22 }
```

Procedure:

1. Class A was created with private member a
2. Inner class B was declared inside A with private member b
3. Method sum() was defined in class B to calculate the sum of a (from outer class) and b (from inner class)
4. Object a of class A was instantiated
5. Object b of inner class B was created using the outer class object a
6. Method sum() was invoked using the inner class object to display the sum of the private members

Output:

Sum: 30

Precautions:

- Inner class object was created using the outer class instance to ensure correct access to outer class members
- Private members were accessed only through the inner class method to maintain encapsulation

Conclusion:

The inner class successfully accessed and used private members of both the outer and inner classes. The sum() method demonstrated how inner classes simplify operations on private data while keeping data encapsulated and secure

Problem 4: Inheritance: Use of super Keyword

Objectives:

- To understand the use of the super keyword in Java for accessing parent class members
- To implement inheritance and demonstrate access to parent class variables, methods, and constructors from a derived class

Theory:

In Java, the super keyword is used in a subclass to refer to the immediate parent class. It allows access to parent class variables, methods, and constructors, even when they are overridden or hidden in the subclass. This is useful for maintaining parent behavior while extending functionality

Code:

```
Java4.java > ...
1  class A {
2      int x;
3
4      A() {
5          x = 10;
6      }
7
8      A(int x) {
9          this.x = x;
10     }
11
12     int getX() {
13         return x;
14     }
15 }
16
17 class B extends A {
18     int y;
19
20     B() {
21         super(100);
22         y = 20;
23     }
24
25     void display() {
26         System.out.println("x from class A: " + super.x);
27         System.out.println("getX() from class A: " + super.getX());
28     }
29 }
30
31 public class Java4 {
32     public static void main(String[] args) {
33         B b = new B();
34         b.display();
35     }
36 }
```

Procedure:

1. Class A was created with member x and two constructors: default and parameterized
2. Method getX() was defined in class A to return the value of x
3. Class B was derived from A using extends
4. Parameterized constructor of A was called in B using super(100)
5. Member x of class A was accessed in class B using super.x
6. Method getX() of class A was called in class B using super.getX()
7. Object b of class B was created and display() method was invoked to print values

Output:

```
getX() from class A: 100
```

Precautions:

- The parameterized constructor of the parent class was called before initializing subclass members
- Parent class members were accessed using super to avoid ambiguity with subclass members

Conclusion:

The super keyword successfully accessed parent class variables, methods, and constructors from the subclass. Inheritance was demonstrated effectively, and super ensured correct interaction between parent and child class members while preserving encapsulation

Problem 5: Interface in Java

Objectives:

- To understand interface implementation and multiple inheritance behavior in Java
- To demonstrate calling interface methods through a class hierarchy

Theory:

In Java, a class can implement multiple interfaces and extend another class simultaneously. Interfaces define abstract methods that must be implemented in the derived class. This allows combining multiple contracts while supporting single class inheritance, providing flexibility without violating Java's restriction on multiple class inheritance

Code:

```
Java5.java > ...
1 interface AI {
2     void PrintA();
3 }
4
5 interface BI {
6     void PrintB();
7 }
8
9 interface CI {
10    void PrintC();
11 }
12
13 // Class A implements AI
14 class A implements AI {
15     public void PrintA() {
16         System.out.println("This is PrintA from Interface AI");
17     }
18 }
19
20 // Class B extends A and implements BI
21 class B extends A implements BI {
22     public void PrintB() {
23         System.out.println("This is PrintB from Interface BI");
24     }
25 }
26
27 // Class C extends B and implements CI
28 class C extends B implements CI {
29     public void PrintC() {
30         System.out.println("This is PrintC from Interface CI");
31     }
32 }
33
34 public class Java5 {
35     public static void main(String[] args) {
36         C obj = new C();
37         obj.PrintA();
38         obj.PrintB();
39         obj.PrintC();
40     }
41 }
42
43 }
```

Procedure:

1. Three interfaces AI, BI, and CI were declared with abstract methods PrintA(), PrintB(), and PrintC() respectively
2. Class A was created to implement interface AI and override PrintA()
3. Class B extended A and implemented interface BI, overriding PrintB()
4. Class C extended B and implemented interface CI, overriding PrintC()
5. Object obj of class C was created
6. Methods PrintA(), PrintB(), and PrintC() were invoked using obj to demonstrate multiple interface method calls

Output:

```
This is PrintA from Interface AI
This is PrintB from Interface BI
This is PrintC from Interface CI
```

Precautions:

- Each interface method was properly implemented in the respective classes to prevent compilation errors
- Correct class hierarchy was maintained to allow seamless access to parent and interface methods

Conclusion:

Multiple interfaces were implemented successfully in a single class hierarchy. Class C was able to access methods from all interfaces via inheritance, demonstrating Java's approach to combining single class inheritance with multiple interface implementation effectively